UNIDAD 6.1

Diseño de Sistemas de Información

Pruebas de software: diseño de casos de prueba, caja negra y caja blanca



INTRODUCCIÓN

Mapa de la unidad

01

Caso de estudio

¿Cómo probarían un programa que clasifica triángulos?

02

Fundamentos

Qué significa probar, psicología, economía y principios.

03

Estrategias

Caja negra y caja blanca: de dónde salen los casos.

04

Caja negra

Equivalencia, valores límite y conjetura de errores.

05

Caja blanca

Sentencias, decisiones y condiciones sobre el código.

06

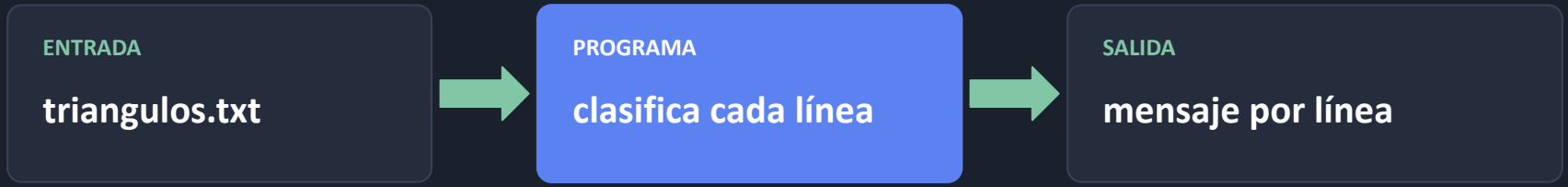
Práctica

Particiones de equivalencia en un cotizador de envíos.

CASO DE ESTUDIO

El problema: clasificar triángulos

El programa lee tres números enteros de un archivo, interpretados como las longitudes de los lados de un triángulo, y escribe un mensaje que informa si el triángulo es escaleno, isósceles o equilátero.




```
3 4 5
3 3 4
2 2 5
```

ejemplo de archivo

```
Línea 1: Triángulo escaleno
Línea 2: Triángulo isósceles
Línea 3: Error: Los lados no forman
         un triángulo válido.
```

salida esperada

CASO DE ESTUDIO

Propiedades de los triángulos



Equilátero

Los tres lados de igual longitud.

$a = b = c$



Isósceles

Dos lados iguales. El lado desigual debe ser menor que la suma de los iguales.

$a = b \cdot 2a > c$



Escaleno

Tres lados desiguales.

$a \neq b \neq c$

Para que exista cualquier triángulo: la suma de los dos lados menores debe ser mayor que el lado mayor (desigualdad triangular).

CASO DE ESTUDIO

Casos de prueba para el triángulo (1/2)

#	Entrada	Qué se prueba	Salida esperada
1	`3, 4, 5`	Escaleno válido	Triángulo escaleno
2	`3, 3, 3`	Equilátero válido	Triángulo equilátero
3	`3, 3, 4`	Isósceles válido	Triángulo isósceles
4	`3, 4, 3` · `4, 3, 3`	Permutaciones del isósceles	Triángulo isósceles
5	`1, 2, 3`	Suma de dos lados = tercero	No forman un triángulo
6	`1, 3, 2` · `2, 3, 1`	Permutaciones del caso 5	No forman un triángulo
7	`12, 15, 30`	Suma de dos lados < tercero	No forman un triángulo
8	`15, 30, 12` · `30, 12, 15`	Permutaciones del caso 7	No forman un triángulo
9	`2, 2, 4`	Isósceles degenerado (2a = c)	No forman un triángulo

Casos de prueba para el triángulo (2/2)

#	Entrada	Qué se prueba	Salida esperada
10	`0, 4, 5`	Un lado nulo (cero)	Error: lados > 0
11	`-3, 4, 5`	Un lado negativo	Error: lados > 0
12	`0, 0, 0`	Todos los lados cero	Error: lados > 0
13	`3, 4.5, 5`	Valor no entero	Formato inválido
14	`3, 4`	Cantidad de valores errónea	Formato inválido

Y la lectura del archivo

- El archivo no existe
- Existe pero está vacío
- El separador es distinto
- Hay letras u otros caracteres
- Números demasiado grandes
- Líneas en blanco

¿Especificaron la salida esperada de cada caso, además de los valores de entrada?

CASO DE ESTUDIO

El código bajo prueba: lectura

```
class Program
{
    static void Main()
    {
        string rutaArchivo = "triangulos.txt";

        if (!File.Exists(rutaArchivo))
        {
            Console.WriteLine($"El archivo '{rutaArchivo}' no existe.");
            return;
        }

        string[] lineas = File.ReadAllLines(rutaArchivo);
        int numeroLinea = 1;

        foreach (string linea in lineas)
        {
            if (string.IsNullOrWhiteSpace(linea))
            {
                Console.WriteLine($"Línea {numeroLinea}: Vacía o inválida");
                numeroLinea++;
                continue;
            }

            string[] partes = linea.Split(' ', StringSplitOptions.RemoveEmptyEntries);
```

1

Existencia

Si el archivo no existe, informa y termina.

2

Línea a línea

Recorre todas las líneas con foreach.

3

Líneas vacías

Una línea en blanco se informa y se continúa.

CASO DE ESTUDIO

El código bajo prueba: validación

```
    if (partes.Length != 3 ||
        !int.TryParse(partes[0], out int a) ||
        !int.TryParse(partes[1], out int b) ||
        !int.TryParse(partes[2], out int c))
    {
        Console.WriteLine($"Línea {numeroLinea}: Formato inválido");
    }
    else
    {
        string resultado = Triangulo.TipoTriangulo(a, b, c);
        Console.WriteLine($"Línea {numeroLinea}: {resultado}");
    }

    numeroLinea++;
}
```

Cuatro condiciones unidas con ||: material para cobertura de condiciones.

Entradas que esta condición decide

- Cantidad de valores distinta de 3
- Texto que no es un entero
- Enteros fuera del rango de Int32
- Separadores distintos del espacio

CASO DE ESTUDIO

El código bajo prueba: clasificación

```
public static string TipoTriangulo(int a, int b, int c)
{
    if (a <= 0 || b <= 0 || c <= 0)
    {
        return "Error: Todos los lados deben ser mayores que cero.";
    }

    if ((a + b <= c) || (a + c <= b) || (b + c <= a))
    {
        return "Error: Los lados no forman un triángulo válido.";
    }

    if (a == b && b == c)
    {
        return "Triángulo equilátero";
    }
    else if (a == b || b == c || a == c)
    {
        return "Triángulo isósceles";
    }
    else
    {
        return "Triángulo escaleno";
    }
}
```

Cinco salidas posibles

1. Lados ≤ 0 ⟶ error
2. No cumple desigualdad ⟶ error
3. Equilátero
4. Isósceles
5. Escaleno

Un caso por rama es el mínimo para cobertura de decisiones.

CASO DE ESTUDIO

Un defecto que los 14 casos no ven

Los 14 casos pasan y el programa igual falla. ¿Qué entrada lo revela?

```
if ((a + b <= c) || (a + c <= b) || (b + c <= a))
{
    return "Error: ...";
}
```

ENTRADA

`2147483647  2147483647  1`

Esperado: isósceles

Obtenido: error

Causa

a + b se desborda en int y da -2, que es menor que c. La suma de dos valores válidos produce un resultado inválido.

Corrección

Promover a long antes de sumar, por ejemplo (long)a + b <= c, o comparar a > c - b con operandos positivos.

¿Qué significa probar?

DEFINICIONES INCORRECTAS

- × Probar es demostrar que no hay errores en el programa.
- × El propósito de probar es mostrar que el programa realiza correctamente las funciones esperadas.
- × Probar es el proceso que lleva a confiar en que el programa hará lo que se supone que debe hacer.

DEFINICIÓN CORRECTA

Probar es el proceso de ejecutar un programa con la intención de encontrar errores.

Es una actividad destructiva: un caso exitoso es el que descubre un error.

Probar no demuestra ausencia de errores

> **“Las pruebas pueden mostrar la presencia de errores, pero nunca su ausencia.”**
>
> Edsger W. Dijkstra (traducción libre)

Meta imposible

Probar que no hay errores no se puede lograr, y las personas rinden peor ante tareas imposibles.

Meta alcanzable

Descubrir errores sí es posible, y orienta el trabajo de prueba.

Una inversión

Un caso de prueba exitoso es el que encuentra un error: cada error hallado ahora es un costo evitado después.

Error, defecto y falla

ERROR (error/mistake)

Equivocación de una persona

Interpretar mal la especificación

DEFECTO (defect/bug/fault)

Está en el código o el diseño

a + b puede desbordar int

FALLA (failure)

Se observa al ejecutar

Un isósceles válido se rechaza

Caso de prueba

Entradas + condiciones de ejecución + resultado esperado. Sin resultado esperado no se puede decidir si hubo falla.

Hay falla en los dos sentidos

Si no hace lo que debe, o si hace lo que no debe: declarar escaleno a (1, 2, 3) también es un error.

La prueba exhaustiva es imposible

¿Es posible probar un programa para encontrar todos los errores? NO.

Psicología de la prueba

¿Quién debería probar?

Economía de la prueba

¿Cuántos casos y para qué?

“Encontrar un error es una inversión, no hacerlo es un gasto.”

Principios de la prueba (1/2)

1	**El resultado esperado es parte del caso**	Se define antes de ejecutar; si no, se tiende a aceptar cualquier salida.
2	**Evitar que el autor pruebe su programa**	Sesgo psicológico y la misma interpretación errónea de la especificación.
3	**La independencia mejora la prueba**	Otra persona o equipo ve lo que el autor no ve. Vale también para empresas.
4	**Inspeccionar cada resultado a fondo**	Muchos errores se observan en la salida y se pasan por alto.
5	**Probar entradas inválidas e inesperadas**	No solo las válidas y esperadas: ahí se esconden muchos errores.

Principios de la prueba (2/2)

6	**Verificar también lo que no debe hacer**	Probar que hace lo esperado es solo la mitad del problema.
7	**Evitar casos de prueba desechables**	Conservarlos y automatizarlos: son una inversión (regresión).
8	**No asumir que no habrá errores**	Planificar el esfuerzo de prueba esperando encontrarlos.
9	**Los errores se agrupan**	Más errores hallados en una sección, más probable que queden otros.
10	**Probar es creativo**	Un desafío intelectual, no una tarea mecánica.

Inspecciones y revisiones



Qué son

Lectura o inspección visual del programa, realizada por un equipo de personas.

Objetivo

Encontrar errores, no soluciones. La corrección se decide después.

Cuándo sirven

Muy útiles en el mantenimiento de un programa.

Más allá de los errores

Feedback al programador y aprendizaje para todo el equipo.

Caja negra y caja blanca

Caja negra

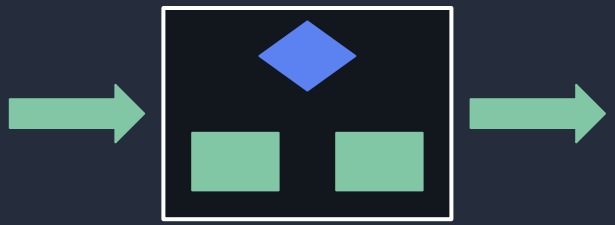
Los casos de prueba derivan solamente de la especificación.



Particiones de equivalencia · Valores límite · Conjetura de errores

Caja blanca

Permiten examinar la estructura interna del programa.



Cobertura de sentencias · de decisiones · de condiciones

ESTRATEGIAS

Diseño de casos de prueba

¿Cuál es el subconjunto de todos los casos posibles con mayor probabilidad de detectar el mayor número de errores?

CAJA NEGRA

1. Particiones de equivalencia
2. Análisis de valores límite
3. Conjetura de errores

CAJA BLANCA

1. Cobertura de sentencias
2. Cobertura de decisiones
3. Cobertura de condiciones

Diseñar casos al azar también es una metodología, aunque muy pobre en resultados.

Un buen caso de prueba

Un buen caso tiene probabilidad razonable de encontrar errores. Como es imposible probar todas las entradas, hay que elegir un subconjunto. Un buen caso tiene dos propiedades:

Propiedad 1

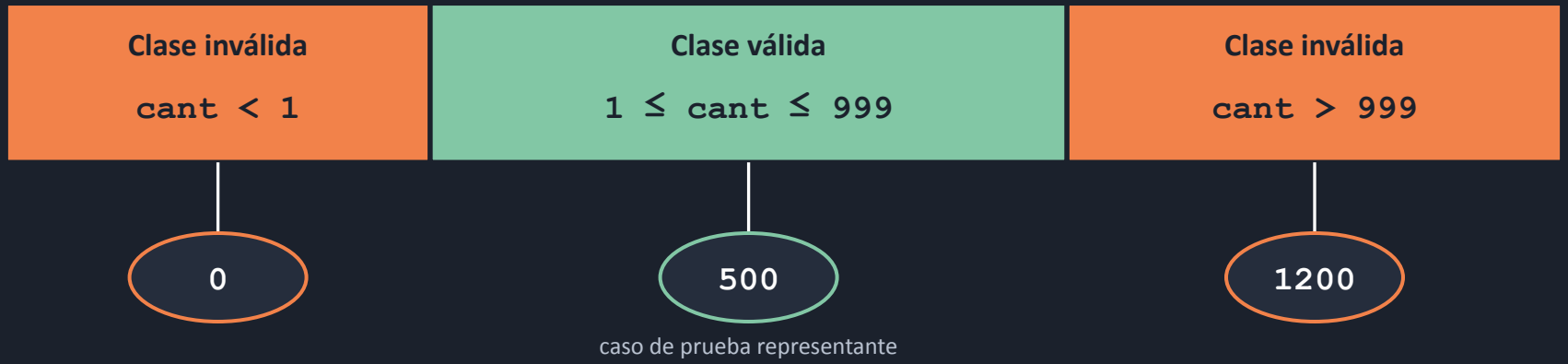
Reduce el número de otros casos de prueba que deben idearse.

Propiedad 2

Cubre un conjunto extenso de otros casos de prueba posibles.

La idea: una clase, un representante

Especificación: la cantidad de ítems debe estar entre 1 y 999.



Si el programa falla con 500, es razonable que falle con cualquier valor de su clase. Un caso representa a toda la clase.

Procedimiento paso a paso

1	**Identificar las condiciones externas de entrada**	Se leen de la especificación.
2	**Partir cada condición en clases**	Clases válidas e inválidas: el dominio queda dividido en un número finito de clases.
3	**Asignar un número único a cada clase**	Permite verificar qué clases cubre cada caso.
4	**Casos válidos: combinar**	Cada nuevo caso cubre tantas clases válidas aún no cubiertas como sea posible, hasta cubrirlas todas.
5	**Casos inválidos: separar**	Cada caso cubre UNA y solo una clase inválida, hasta cubrirlas todas.

Cómo identificar las clases

Si la condición de entrada es…	Clases válidas	Clases inválidas	Ejemplo
Un rango de valores	1	2	Cantidad entre 1 y 999
Una cantidad específica de valores	1	2	Tres lados (menos de 3 y más de 3)
Un conjunto de valores tratados distinto	Una por cada valor	1	Tipo de libro: cartón, cartulina, tela
Una condición “debe ser”	1	1	El primer carácter debe ser una letra

Subdividir una clase cuando el programa trata distinto a algunos de sus elementos.

Una sola clase inválida por caso

Especificación: ingresar tipo de libro (cartón, cartulina, tela) y cantidad (1–999).

INCORRECTO

`XXX 0`

Cubre dos clases inválidas a la vez. Si el programa valida el primer dato y sale por tipo inválido, el control de cantidad nunca se activa.

CORRECTO

`XXX 10`

`cartón 0`

Un caso por clase inválida, con el resto de los datos válidos.

Para que un control sea probado, el resto de los datos debe ser válido.

Ejemplo: tabla de clases

Condición externa	Clase válida	Clases inválidas
La cantidad de ítems debe estar entre 1 y 999	(1) $1 \leq cant \leq 999$	(2) $cant < 1$ (3) $cant > 999$
El primer carácter debe ser una letra	(4) Palabra que comienza con una letra	(5) Palabra que comienza con un número

Casos de prueba resultantes

Válido: (500, “Ana”) cubre (1) y (4) en un solo caso
Inválido: (0, “Ana”) cubre (2) · (1000, “Ana”) cubre (3) · (500, “7up”) cubre (5)

Volvamos al triángulo

CLASES VÁLIDAS

1. Tres enteros > 0 que forman un equilátero
2. … que forman un isósceles
3. … que forman un escaleno

Cada una puede subdividirse: por ejemplo, las permutaciones de los lados iguales.

CLASES INVÁLIDAS

4. Algún lado ≤ 0
5. Dos lados suman igual que el tercero
6. Dos lados suman menos que el tercero
7. Valor no entero
8. Cantidad de valores distinta de 3

Análisis de valores límite

Dominio: 1 a 255



- Los casos que exploran los límites rinden más que los que no lo hacen.
- Los límites son los valores justo encima y justo debajo de los márgenes de cada clase, de entrada y de salida.
- Cada margen de cada clase debe ser probado.

Valores límite: ejemplos

Dominio -1,0 a 1,0

- `-1,0`
- `1,0`
- `-1,001`
- `1,001`

Dos valores en el borde y dos inmediatamente fuera.

Dominio 1 a 255

- `1`
- `255`
- `0`
- `256`

Un entero: el valor fuera es el borde ±1.

Contribución mensual de $0,00 a $500.000

- `$0`
- `$500.000`
- `negativa`
- `mayor a $500.000`

Los límites también se buscan en la salida calculada.

Conjetura de errores

Hay personas con habilidad natural para probar: intuyen, por experiencia, qué errores son probables y diseñan casos que los revelen.

Función de clasificación de una lista

1. La lista está vacía
2. Contiene una sola entrada
3. Todos los datos tienen el mismo valor
4. Ya está clasificada

Lista de sospechosos habituales

- Vacío, nulo o solo espacios
- Cero, negativos y valores máximos
- Caracteres especiales y acentos
- Fechas límite (29 de febrero)
- Duplicados y orden inverso

Cobertura de sentencias

Verifica que cada línea (sentencia) se ejecute al menos una vez durante las pruebas.

```
void evaluar(int x) {
    if (x > 0) {
        Console.WriteLine("Positivo");
    }
    Console.WriteLine("Fin");
}
```

Con x = 1 se ejecutan todas las líneas: 100 %. Nunca se probó x ≤ 0.

```
if (usuario.IsAdmin || usuario.Id ==
1)
{
    realizarAccionPeligrosa();
}
```

IsAdmin == true ejecuta la línea, pero no prueba qué pasa con Id == 1 e IsAdmin == false.

Una prueba de cobertura de sentencias es débil.

Cobertura de decisiones

Cada decisión lógica (if, while, for…) debe tomar ambos valores, verdadero y falso, al menos una vez.

```
void evaluar(int x) {
    if (x > 0)
        Console.WriteLine("Positivo");
    Console.WriteLine("Fin");
}
```

Caso	x > 0	Salida
1	verdadero	Positivo, Fin
2	falso	Fin

2 casos: cobertura de decisiones del 100 %.

Las dos ramas (verdadera y falsa) quedan ejercitadas, y con ellas todas las sentencias.

Cobertura de decisiones

Cada decisión lógica (if, while, for…) debe tomar ambos valores, verdadero y falso, al menos una vez.

```
if (a > 0 && b > 0) // decisión: (a > 0 && b > 0)
```

Caso	Input	Salida
1	(a=1, b=1)	Positivo
2	(a=-1, b=1)	Negativo

2 casos: cobertura de decisiones del 100 %.

Cobertura de condiciones

Cada condición individual de una expresión compuesta debe evaluarse en verdadero y en falso, sin importar el resultado final de la decisión.

```
void evaluar(int a, int b) {
    if (a > 0 || b > 0)
        Console.WriteLine("Al menos uno es
positivo");
}
```

a	b	a > 0	b > 0	Decisión
1	-1	V	F	V
-1	1	F	V	V

Dos condiciones: (1) a > 0 (2) b > 0

Cada condición toma V y F, pero la decisión fue siempre verdadera: nunca se probó la rama falsa.

Comparando los criterios

Criterio	Exige	¿Garantiza lo anterior?
Sentencias	Cada línea se ejecuta al menos una vez	Es el más débil
Decisiones	Cada decisión toma V y F	Incluye sentencias
Condiciones	Cada condición toma V y F	No siempre cubre decisiones
Condición/decisión	Ambos criterios a la vez	Incluye los dos anteriores

En (a > 0 || b > 0), los casos (1, 1) y (-1, -1) alcanzan condición/decisión. Una cobertura del 100 % no prueba que no falte funcionalidad: se combina con técnicas de caja negra.

PRÁCTICA

Cotizador de envíos

Aplicar particiones de equivalencia a una especificación nueva

PRÁCTICA

Especificación del cotizador

El programa recibe peso, tipo de servicio y código postal de destino, y devuelve el costo de un envío o un mensaje de error que indica el dato inválido.

Dato	Regla
Peso (kg)	Número con un decimal, de 0,1 a 30,0 inclusive.
Servicio	Texto exacto: Estándar, Express o Económico.
Código postal	Entero de 4 dígitos, de 1000 a 9999.

Tarifa

Base: liviano (hasta 5,0 kg) $3.000
pesado (más de 5,0 kg) $6.000

Multiplicador: Económico × 0,8
Estándar × 1 · Express × 1,5

Ejemplo: 2,5 kg, Estándar, 5000 → Costo: $3.000

PRÁCTICA

Consigna

1. Listen las condiciones externas de entrada.
2. Armen la tabla de clases de equivalencia, válidas e inválidas, con numeración única.
3. Diseñen la menor cantidad posible de casos válidos que cubran todas las clases válidas.
4. Diseñen un caso por cada clase inválida.
5. Escriban la salida esperada de cada caso.

Para discutir

- ¿Cuántos casos válidos son el mínimo?
- ¿Por qué no combinar dos clases inválidas en un caso?
- ¿Qué dudas de la especificación encontraron?

Entregable: tabla de clases + tabla de casos con salida esperada.

SOLUCIÓN

Solución: clases de equivalencia

Condición	Clases válidas	Clases inválidas
Peso (kg)	(1) 0,1 ≤ peso ≤ 5,0 liviano (2) 5,1 ≤ peso ≤ 30,0 pesado	(3) peso < 0,1 (4) peso > 30,0 (5) no numérico
Servicio	(6) Estándar (7) Express (8) Económico	(9) otro texto (10) vacío
Código postal	(11) 1000 ≤ cp ≤ 9999	(12) cp < 1000 (13) cp > 9999 (14) no numérico

El peso se subdivide en liviano y pesado porque el programa los trata distinto (tarifa base).

SOLUCIÓN

Solución: casos válidos

Caso	Peso	Servicio	CP	Clases	Salida esperada
V1	2,5	Estándar	5000	1, 6, 11	Costo: $3.000
V2	12,0	Express	1425	2, 7, 11	Costo: $9.000
V3	20,0	Económico	2000	2, 8, 11	Costo: $4.800

Cálculo de salidas

V1: liviano $3.000 × 1 · V2: pesado $6.000 × 1,5 · V3: pesado $6.000 × 0,8. Cualquier combinación que cubra las 6 clases válidas con 3 casos es correcta.

SOLUCIÓN

Solución: casos inválidos

Caso	Peso	Servicio	CP	Clase	Salida esperada
I1	0,0	Estándar	5000	3	Error: peso inválido
I2	35,0	Estándar	5000	4	Error: peso inválido
I3	abc	Estándar	5000	5	Error: peso inválido
I4	2,5	Premium	5000	9	Error: servicio inválido
I5	2,5	(vacío)	5000	10	Error: servicio inválido
I6	2,5	Estándar	999	12	Error: código postal inválido
I7	2,5	Estándar	10000	13	Error: código postal inválido
I8	2,5	Estándar	ABCD	14	Error: código postal inválido

SOLUCIÓN

Errores frecuentes y siguiente paso

Errores frecuentes

- Combinar dos clases inválidas en un caso
- Olvidar las clases “vacío” y “no numérico”
- No subdividir peso en liviano y pesado
- Usar siempre el mismo servicio en los casos válidos
- No escribir la salida esperada

Siguiente paso: valores límite

Peso

`0,0 · 0,1 · 5,0 · 5,1 · 30,0 · 30,1`

Código postal

`999 · 1000 · 9999 · 10000`

Ambigüedad a detectar: ¿“Express” acepta mayúsculas o minúsculas? La especificación no lo dice.

CIERRE

Para llevarse

1

Probar es buscar errores

Un caso exitoso es el que encuentra un error.

2

No se puede probar todo

Hay que elegir bien: técnica antes que azar.

3

Clases de equivalencia

Un representante por clase; válidos combinados, inválidos uno por vez.

4

Valores límite

Los errores se esconden en los bordes de las clases.

5

Cobertura lógica

Sentencias < decisiones; las condiciones se suman.

6

Siempre, la salida esperada

Sin resultado esperado no hay caso de prueba.